

LeetCode一百题解析

这些都是我个人的理解和经验，毕竟我也是个新手，做这些都是为了更好的理解，如果大家发现有什么问题，欢迎指出改正，谢谢大家

1.两数之和

题目

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** `target` 的那 **两个** 整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。

你可以按任意顺序返回答案。

示例 1:

```
1 输入: nums = [2,7,11,15], target = 9
2 输出: [0,1]
3 解释: 因为 nums[0] + nums[1] == 9，返回 [0, 1]。
```

示例 2:

```
1 输入: nums = [3,2,4], target = 6
2 输出: [1,2]
```

示例 3:

```
1 输入: nums = [3,3], target = 6
2 输出: [0,1]
```

提示:

- `2 <= nums.length <= 104`
- `-109 <= nums[i] <= 109`
- `-109 <= target <= 109`
- 只会存在一个有效答案

进阶: 你可以想出一个时间复杂度小于 `O(n2)` 的算法吗？

解析

这道题目的目的是从给你的一个数组里面找到两个数，这两个数的和要等于目标值。

这样就很清晰我们的思路了

我们可以遍历数组，将遍历的每一个值都与目标值做差，得到的这个值，如果存在于数组中，那么这两个数就满足题目的要求。

下面使用C++实现

```
1 vector<int> twoSum(vector<int>& nums, int target) {
2     int n = nums.size();
3     for (int i = 0; i < n; i++) {
4         for (int j = i + 1; j < n; j++) {
5             if (nums[i] + nums[j] == target) {
6                 return {i, j};
7             }
8         }
9     }
10    return {};
11 }
```

这个解法是典型的暴力循环解法，能解出来，但是时间复杂度为 $O(n^2)$ 。

但这个仍然有优化空间，就是使用哈希表

```
1 vector<int> twoSum(vector<int>& nums, int target) {
2     unordered_map<int, int> map; // key: 数值, value: 下标
3
4     for (int i = 0; i < nums.size(); i++) {
5         int need = target - nums[i]; // 需要找的另一个数
6
7         // 如果哈希表里已经有这个数，直接返回结果
8         if (map.find(need) != map.end()) {
9             return {map[need], i};
10        }
11
12        // 没有就把当前数和下标存进哈希表
13        map[nums[i]] = i;
14    }
15
16    return {}; // 题目保证有解，这里不会执行
17 }
18
```

使用哈希表，可以将时间复杂度从 $O(n^2)$ 降到 $O(n)$

这也说明了哈希表是一个对于算法来说很重要的东西，要重点掌握其使用方式，接下来，我们先讲一下这个方法下的代码。

首先，使用`unordered_map<int, int>` hash来生成定义一个哈希表，`<>`里的两个参数分别代表的是**key（键）**的类型和**value（值）**的类型，就如同这里，使用的是**int**类型的**key**和**value**，因为我们要存放的对象是数组的下标和对应的值。

然后我们遍历数组对遍历的每一个值进行计算，算出另外一个符合的值，然后在哈希表里面查询这个值是否存在。如果有就返回，没有就继续找。

这里说一下这个地方

```
1 if (map.find(need) != map.end()) {
2     return {map[need], i};
3 }
```

`map.find(key)`是根据key值来查找的，返回的是一个迭代器，是一个指向{key, value}的一个“指针”，不仅可以像如上代码使用`map[need]`来取出这个key下对应的value，也可以采用如下办法，

```
1 auto it = map.find(2);
2 // 拿到 key
3 cout << it->first;    // 输出 2
4 // 拿到 value
5 cout << it->second;  // 输出 0
```

如果没找到的话，它就会返回`map.end()`，这实际就是一个哈希表结束的标记，如果没找到，`map.find()`就会返回这个标记，然后和`map.end()`相互比较，如果相等就是没找到，不等就是找到了。

为什么时间复杂度会降低到 $O(1)$ 呢，这是因为哈希表的特性了，在哈希表里面找东西，不是用遍历的查找，而是直接锁定到目标位置，而这个直接锁定是怎么实现的呢，是算出来的，这里就不多提及了，感兴趣的可以去单独看看。

这里也简单说一下哈希的使用场景

哈希的核心优势是 $O(1)$ 时间复杂度的增删查，是解决“快速查找、去重、计数”类问题的首选工具，主要分为以下几类场景：

快速去重与存在性判断

- **典型场景：**判断数组 / 字符串中是否存在重复元素、元素是否出现过、过滤重复数据。
- **代表题目：**最长连续序列、存在重复元素、有效的字母异位词。
- **核心用法：**用 `unordered_set` 存储元素，自动去重，`find / count` 操作可 $O(1)$ 判断元素是否存在。

频率统计与计数

- **典型场景：**统计元素出现次数、字符频率、数组中数字的出现次数。
- **代表题目：**两数之和、字母异位词分组、多数元素、字符串中字符出现次数统计。
- **核心用法：**用 `unordered_map` 记录 key-value（元素 - 次数），可快速查询和更新计数。

快速映射与关联查找

- **典型场景：**需要建立元素间的——映射关系，快速根据一个元素找到对应的另一个元素。
- **代表题目：**两数之和（存差值）、罗马数字转整数、用户 ID 与数据的关联查询。
- **核心用法：**用 `unordered_map` 存储映射关系， $O(1)$ 时间完成查找。

缓存 / 临时存储，避免重复计算

- **典型场景：**动态规划 / 递归中存储已计算的结果（记忆化搜索）、临时存储遍历过的节点 / 状态。
- **代表题目：**带记忆化的斐波那契数列、图的遍历中记录访问过的节点、回溯剪枝。
- **核心用法：**用哈希表存储已计算的结果，避免重复计算，提升效率。

2.字母异位词分组

题目

给你一个字符串数组，请你将 字母异位词 组合在一起。可以按任意顺序返回结果列表。

示例 1:

输入: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

输出: [["bat"],["nat","tan"],["ate","eat","tea"]]

解释:

- 在 strs 中没有字符串可以通过重新排列来形成 "bat"。
- 字符串 "nat" 和 "tan" 是字母异位词，因为它们可以重新排列以形成彼此。
- 字符串 "ate" , "eat" 和 "tea" 是字母异位词，因为它们可以重新排列以形成彼此。

示例 2:

输入: strs = [""]

输出: [[""]]

示例 3:

输入: strs = ["a"]

输出: [["a"]]

提示:

- `1 <= strs.length <= 104`
- `0 <= strs[i].length <= 100`
- `strs[i]` 仅包含小写字母

解析

还是一样，我们先来理解字母异位词这个概念。

就是说，这个字符串数组里面的这些字符串，能够通过任意的组合形成彼此的字符串样式，例如“abc”，“cba”，这两个字符串都可以通过变换变成彼此，然后满足这个需求的就可以分为一组。

说人话，就是拥有相同的字母类型和对应个数的字符串就可以分为一组，例如两个字符串里面拥有相同数量的a，相同数量的b，相同数量的c，那么就可以认为这是满足的。

所以我们就有了一个基本思路，对每个字符串进行排序，这样就可以将满足的字符串变成长得一模一样的字符串，然后将满足的字符串下标进行分组，再将这些重新输出为二维数组就可以了。

下面使用C++进行实现：

```
1 vector<vector<string>> groupAnagrams(vector<string>& strs) {
2     //这个地方我们先创建两个动态数组，一个用来存放我们返回的结果，一个存放目标字符串是否已
    被分组的布尔值
3     vector<vector<string>> res;
4     vector<bool> used(strs.size(), false);
```

```

5      //这里开始进行遍历
6      for (int i = 0; i < strs.size(); i++) {
7          if (used[i]) continue; //这里意思是如果已经使用了就直接跳过，不再进行无用的遍历
8
9          //这里创建一个动态的字符串数组来分组，满足的就分到这一组里面
10         vector<string> group;
11         group.push_back(strs[i]);
12         used[i] = true; //标记为已使用
13
14         // 拿到当前字符串排序
15         string key_i = strs[i];
16         sort(key_i.begin(), key_i.end());
17
18         // 遍历后面所有字符串
19         for (int j = i + 1; j < strs.size(); j++) {
20             if (used[j]) continue;
21
22             string key_j = strs[j];
23             sort(key_j.begin(), key_j.end());
24
25             // 如果排序后一样 → 同一组
26             if (key_i == key_j) {
27                 group.push_back(strs[j]);
28                 used[j] = true;
29             }
30         }
31         res.push_back(group);
32     }
33     return res;
34 }

```

这个仍然采用的是暴力的方式，在我看来，只要你会写暴力的解，那么在其上进行优化也不会太难了。

这个暴力的解法你在LeetCode上提交的话会执行不通过，因为时间超了，时间复杂度为 $O(n^2 \times k \log k)$ ，所以我们可以采取优化，这里我们也可以发现和上一题一样的使用双for来循环对照，所以我们这个题也可以用哈希表来将第二个for给优化了。

下面我们就来使用哈希表来解决

```

1  vector<vector<string>> groupAnagrams(vector<string>& strs) {
2
3      // 哈希表
4      // key: 排序后的字符串
5      // value: 同一组的所有原字符串
6      unordered_map<string, vector<string>> map;
7
8      // 1. 遍历一次所有字符串 O(n)
9      for (string s : strs) { //这里的用法就是遍历strs数组，再把里面的每一个放到s
10         // 2. 排序生成 key
11         string key = s;
12         sort(key.begin(), key.end());
13
14         // 3. 直接放进哈希表对应组里 O(1)
15         map[key].push_back(s);
16     }

```

```

17
18         // 4. 把哈希表的结果转成二维数组输出
19         vector<vector<string>> res;
20         for (auto& entry : map) { // 这里auto就是自动分类型，map里是啥就是啥类型，然后
            &就是取引用嘛，会比复制一份出来快一些
21             res.push_back(entry.second);
22         }
23
24         return res;
25     }

```

优化完后，时间复杂度也降低到了 $O(n \times k \log k)$

同时我们也发现了代码也会少一些，这就是算法优化的好处

3.最长连续序列

题目

给定一个未排序的整数数组 `nums`，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。

请你设计并实现时间复杂度为 $O(n)$ 的算法解决此问题。

示例 1:

```

1  输入: nums = [100,4,200,1,3,2]
2  输出: 4
3  解释: 最长数字连续序列是 [1, 2, 3, 4]。它的长度为 4。

```

示例 2:

```

1  输入: nums = [0,3,7,2,5,8,4,6,0,1]
2  输出: 9

```

示例 3:

```

1  输入: nums = [1,0,1,2]
2  输出: 3

```

提示:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

解析

先来看题目，在一个没有排序的整数数组里面找出数字的连续最长序列。

数字连续序列是啥呢？

不难理解如0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 这样的数字连续就是这个所谓的连续序列。

好了我们知道了要做什么之后，再看题目要求说实现的时间复杂度为 $O(n)$,这就说明了它不要你用排序的方法，因为排一次序的时间复杂度为 $O(k\log k)$,所以不能用

那么，根据前面几道题的思路，我们依然采用哈希的方式来解决，只不过这里使用的是哈希集合。

```
1  int longestConsecutive(vector<int>& nums) {
2      // 哈希集合：去重 + 快速查找,这里去重说的是哈希集合的自动去重，因为哈希集合不允许
   重复元素
3      //所以在构造哈希集合时会自动去重。
4      unordered_set<int> num_set(nums.begin(), nums.end());
5      int max_len = 0;
6
7      // 遍历哈希集合中的每个元素
8      for (int num : num_set) {
9          // 关键：只处理连续序列的起点（num-1 不存在）
10         //这里这么做的原因是为了避免重复计算连续序列的长度，如果 num-1 存在，
11         //那么说明 num 不是连续序列的起点，已经在之前的迭代中被计算过了。
12         //如[1,2,3,4,5,6],当迭代到 num=2 时，num-1=1 存在，所以跳过计算以 2 开
   头的连续序列
13         //这是因为在以1为起点的时候就算过一遍了。
14         if (num_set.find(num - 1) == num_set.end()) {
15             int current_num = num;
16             int current_len = 1;
17
18             // 向后查找连续数字
19             while (num_set.find(current_num + 1) != num_set.end()) {
20                 current_num++;
21                 current_len++;
22             }
23
24             // 更新最长长度
25             max_len = max(max_len, current_len);
26         }
27     }
28     return max_len;
29 }
```

这里有人会有疑问了，有for和while，这是两个循环，怎么时间复杂度仍然是 $O(n)$ 呢？

我们来举例子说明，例如[1,2,3,5,6,7,8]

在for里，我们以1为起点的时候，访问一次1。

然后在while里面，我们各自访问了一遍后面的连续的数，2，3，这也各是访问一次。

然后就发现这个序列连不上了，又跳回了for。

但是在if的地方截胡了，2，3跳过了。

那么就来到了5，和上面一样的道理，5访问了一次，

然后6, 7, 8在while里各一次。

发现了什么？

一个集合在这样的for和while下相当于只遍历了一遍。

又有人会问，不是执行了一个if吗，为啥不算执行呢？

不得不说if 判断确实每轮都会执行，但它只是一次极快的条件检查，不会重复遍历、不会重复处理元素；真正消耗时间的是 while 循环的连续查找，而所有元素**只会被 while 访问唯一一次**，不会重复遍历。for 只是全局遍历一遍所有数，后续非起点的元素仅做一次 if 判断就直接跳过，没有额外冗余操作，整体所有元素总共只被完整遍历一轮，所以即便存在 for 嵌套 while，整体时间复杂度依旧是O(n)。

4.移动零

题目

给定一个数组 `nums`，编写一个函数将所有 `0` 移动到数组的末尾，同时保持非零元素的相对顺序。

请注意，必须在不复制数组的情况下原地对数组进行操作。

示例 1:

```
1 输入: nums = [0,1,0,3,12]
2 输出: [1,3,12,0,0]
```

示例 2:

```
1 输入: nums = [0]
2 输出: [0]
```

提示:

- `1 <= nums.length <= 104`
- `-231 <= nums[i] <= 231 - 1`

进阶：你能尽量减少完成的操作次数吗？

解析

题目给了我们一个数组，然后把所有的0都放到数组的末尾后，其余非零的数的原本顺序不能变，还不能使用数组赋值的方法。

既然要求0置后，那么我们为何不管0，直接操作其它的数，把这些数按照原有的顺序全部挤到前面去，最后把空余的置为0不就行了吗。

这样，我们可以使用双指针，来对其进行操作。

下面是代码

```
1 void moveZeroes(vector<int>& nums) {
2     int n = nums.size();
```

```

3         int i = 0; //这个指针用来记录非零元素应该放置的位置
4         for(int j = 0; j < n; j++){
5             //j指针用来遍历数组，找到非零元素
6             if(nums[j] != 0){
7                 nums[i] = nums[j];
8                 i++;
9             }
10        }
11        //将剩余的位置填充为0
12        for(int k = i; k < n; k++){
13            nums[k] = 0;
14        }
15    }

```

除了这个，我们还可以使用交换的方式

```

1 void moveZeroes(vector<int>& nums) {
2     int slow = 0;
3     for (int fast = 0; fast < nums.size(); fast++) {
4         if (nums[fast] != 0) {
5             swap(nums[slow], nums[fast]);
6             slow++;
7         }
8     }
9 }

```

这里也可以简单说一下双指针的一些内容。

双指针主要分两类：

1. **同向双指针（快慢 / 滑动窗口）**：两个指针同方向移动，用于原地筛选 / 移动元素（如移动零、去重）、滑动窗口求子串 / 子数组（如最长无重复子串）、链表找环 / 中点。核心是用一个指针筛选，另一个记录有效位置，实现 $O(n)$ 时间、 $O(1)$ 空间。
2. **相向双指针（对撞）**：两个指针从两端向中间移动，多用于有序数组的查找（如两数之和）、回文判断。利用有序性，通过和 / 值的大小调整指针，高效缩小范围。

5.盛最多水的容器

题目

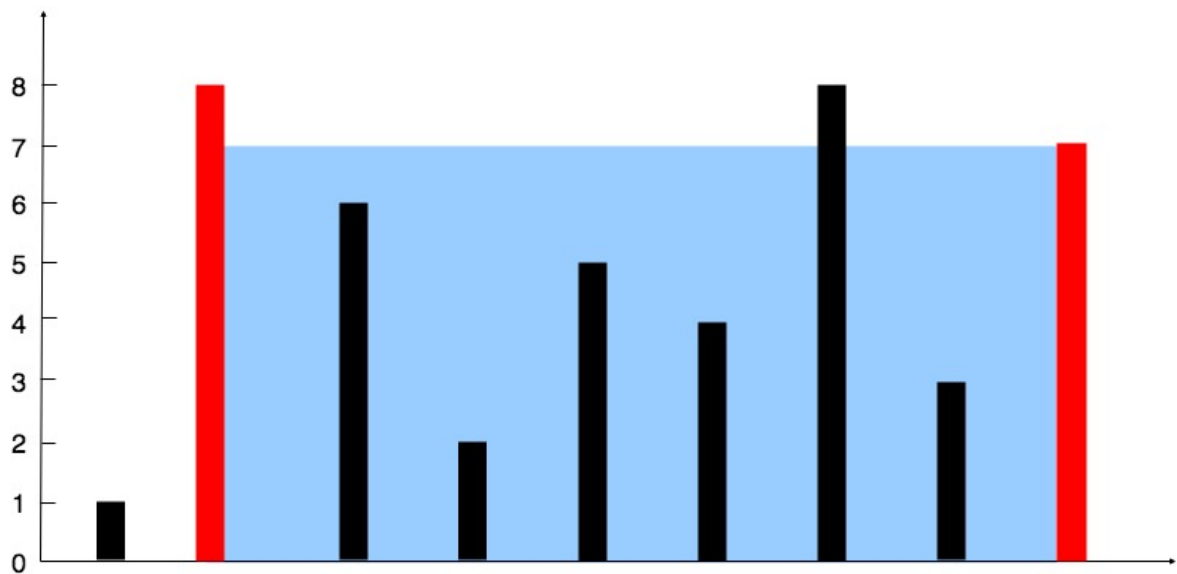
给定一个长度为 n 的整数数组 `height`。有 n 条垂线，第 i 条线的两个端点是 $(i, 0)$ 和 $(i, height[i])$ 。

找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。

返回容器可以储存的最大水量。

说明：你不能倾斜容器。

示例 1：



```
1 输入: [1,8,6,2,5,4,8,3,7]
2 输出: 49
3 解释: 图中垂直线代表输入数组 [1,8,6,2,5,4,8,3,7]。在此情况下, 容器能够容纳水 (表示为蓝色部分) 的最大值为 49。
```

示例 2:

```
1 输入: height = [1,1]
2 输出: 1
```

提示:

- `n == height.length`
- `2 <= n <= 105`
- `0 <= height[i] <= 104`

解析

依旧我们来看题目的要求, 说大白话就是找到这个容器的两个边要让里面含的水越多越好, 如上题里面提到的双指针, 我们就要用到其中的相向双指针, 从两边往中间找。

下面是代码实现

```
1 int maxArea(vector<int>& height) {
2     //定义我们的左右指针, 分别指向数组的两端
3     int left = 0;
4     int right = height.size() - 1;
5     //我们要找的最大值
6     int maxArea = 0;
7     //开始循环, 尝试每一种可能的最大情况
8     while(left < right){
9         //以短边为准算容量
10        int area = min(height[left], height[right]) * (right - left);
11        //将更大的赋值回去
12        maxArea = max(maxArea, area);
13        //这里是在调整边界, 哪边矮了就调整哪一边, 这样才有可能尝试到最大的
```

```

14         if(height[left] < height[right]){
15             left++;
16         }
17         else{
18             right--;
19         }
20     }
21     return maxArea;
22 }

```

6.三数之和

题目

给你一个整数数组 `nums`，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 `i != j`、`i != k` 且 `j != k`，同时还满足 `nums[i] + nums[j] + nums[k] == 0`。请你返回所有和为 0 且不重复的三元组。

注意：答案中不可以包含重复的三元组。

示例 1：

```

1  输入：nums = [-1,0,1,2,-1,-4]
2  输出：[[-1,-1,2],[-1,0,1]]
3  解释：
4  nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0 。
5  nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0 。
6  nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0 。
7  不同的三元组是 [-1,0,1] 和 [-1,-1,2] 。
8  注意，输出的顺序和三元组的顺序并不重要。

```

示例 2：

```

1  输入：nums = [0,1,1]
2  输出：[]
3  解释：唯一可能的三元组和不为 0 。

```

示例 3：

```

1  输入：nums = [0,0,0]
2  输出：[[0,0,0]]
3  解释：唯一可能的三元组和为 0 。

```

提示：

- `3 <= nums.length <= 3000`
- `-105 <= nums[i] <= 105`

解析

三数之和是两数之和的经典升级版，解题本质一脉相承，只是求和对象由两个数字升级为三个数字。

两数之和常规采用**哈希表**快速匹配目标值，时间效率极高；但**三数之和不适合直接使用哈希表**，原因如下：

1. 两数之和只需寻找唯一组合，无需去重；而三数之和要求返回**无重复三元组**，哈希表很难高效处理多层重复元素，去重逻辑极其繁琐，容易产生大量重复结果。
2. 哈希表适合无序数组单层查找，三数之和需要同时约束三个下标互不相同，用哈希表嵌套判断会增加复杂逻辑，代码冗余、易出错。

因此本题放弃纯哈希表思路，改用**排序 + 双指针**最优解法：

先固定第一个元素，把「三数之和为 0」转化为**剩余两数之和等于该数相反数**的两数之和子问题；再借助有序数组的特性，用左右双指针动态收缩查找合法组合，配合相邻重复元素跳过的去重操作，既保证时间复杂度可控，又能天然规避重复三元组。

```
1  vector<vector<int>> threeSum(vector<int>& nums) {
2      vector<vector<int>> res;
3      // 先排序，方便去重和双指针
4      sort(nums.begin(), nums.end());
5      int n = nums.size();
6
7      //这里是三数之和的核心逻辑，外层循环固定一个数，内层使用双指针寻找剩余两个数的和为
      //固定数的相反数
8      for (int i = 0; i < n - 2; ++i) {
9          // 去重: i 和上一个数相同，跳过,因为上一个数已经处理过了，避免重复结果
10         //这里说的重复结果的意思是题目里说的，不管顺序，三元组不能重复，就是不同的三
            //元组里面所含有的数字不能重复
11         //和顺序无关，所以如果当前数字和上一个数字相同，就跳过，避免产生重复的三元组
            //结果
12         if (i > 0 && nums[i] == nums[i - 1]) continue;
13
14         // 双指针，寻找剩余两个数的和为 -nums[i]
15         int j = i + 1;
16         int k = n - 1;
17         int target = -nums[i];
18
19         while (j < k) {
20             int sum = nums[j] + nums[k];
21             if (sum == target) {
22                 res.push_back({nums[i], nums[j], nums[k]});
23                 // j 去重
24                 while (j < k && nums[j] == nums[j + 1]) j++;
25                 // k 去重
26                 while (j < k && nums[k] == nums[k - 1]) k--;
27                 j++;
28                 k--;
29             }
30             else if (sum < target) {
31                 j++; // 和太小，左指针右移
32             }
33             else {
34                 k--; // 和太大，右指针左移
```

```

35         }
36     }
37 }
38 return res;
39 }

```

7.接雨水

题目

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1:



```

1  输入: height = [0,1,0,2,1,0,1,3,2,1,2,1]
2  输出: 6
3  解释: 上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

```

示例 2:

```

1  输入: height = [4,2,0,3,2,5]
2  输出: 9

```

提示:

- $n == \text{height.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

解析

其实这道题很好理解是什么意思，就是说那些又柱子组成的空隙里面能装多少水

下面是第一种思路的解法：

```

1  int trap(vector<int>& height) {
2      int res = 0;
3      // 找到最高柱子，决定有多少层
4      int max_h = *max_element(height.begin(), height.end());

```

```

5
6    // 逐层计算
7    for (int level = 1; level <= max_h; ++level) {
8        bool has_left = false;
9        int temp = 0;
10
11        for (int h : height) {
12            if (h >= level) {
13                // 遇到柱子，把前面积累的水加上
14                has_left = true;
15                res += temp;
16                temp = 0;
17            } else if (has_left) {
18                // 左边有柱子，当前位置可以存水
19                temp++;
20            }
21        }
22    }
23    return res;
24 }

```

这是我第一时间想到的解决方法，就是按层计数。

按层法核心就是把整个柱状图横向一层层切开，从最底层到最高柱子的高度逐层遍历。每一层里从左往右扫描所有柱子，先找到第一个能挡住当前层的左柱子，在这之后出现的空白位置都可以存水，继续往后走，每碰到一根新的柱子，就把这一段中间空白的数量算作当前层接住的雨水，再重新统计下一段空白。一层层全部算完，把每一层的蓄水量累加起来，就是整体一共能接住的雨水总量。

但这个就是有很明显的局限性了。

按层法的时间复杂度为 $O(n \times \max H)$ ， n 是柱子数量， $\max H$ 为最大柱子高度，需要依据最大高度逐层遍历整个数组，存在大量重复扫描；该方法最大局限是极易受柱子最大高度影响，当柱子高度数值极大时，运算次数会急剧增加、运行效率大幅下降，容易超时，无法处理大规模数据，不能满足算法题的性能要求。

所以我们还是要用双指针来优化。

```

1  int trap(vector<int>& height) {
2      int left = 0;
3      int right = height.size() - 1;
4      int leftMax = 0;
5      int rightMax = 0;
6      int res = 0;
7
8      while (left < right)
9      {
10         //左边的墙小于右边的墙才有可能有水的积蓄
11         if (height[left] < height[right])
12         {
13             //如果左指针指向的这个墙比左边的最高墙还高，那么就更新左边的最高墙
14             if (height[left] >= leftMax)
15                 leftMax = height[left];
16             //同样的道理，如果矮了，就说明这地方可以积水
17             else
18                 res += leftMax - height[left];
19             left++;

```

```

20     }
21     //右边的道理和左边的一样
22     else
23     {
24         if (height[right] >= rightMax)
25             rightMax = height[right];
26         else
27             res += rightMax - height[right];
28         right--;
29     }
30 }
31 return res;
32 }

```

总而言之，双指针法从数组**左右两端**向中间遍历，分别维护左侧、右侧已遍历区域的最高柱子高度。每次比较左右指针指向的柱子高度，**较矮一侧的柱子所在位置可直接计算接水量**（该位置的积水由同侧最高柱子决定，另一侧必然有更高柱子挡水）；若当前柱子高于同侧最高高度，则更新最大高度，计算完后将较矮一侧的指针向中间移动，直到两指针相遇，累加所有位置的接水量即为结果。

并且两个指针全程只向中间移动，**最多遍历数组一次**，没有嵌套循环，执行次数与数组长度成正比，时间复杂度减小到了 $O(n)$

8.无重复字符的最长子串

题目

给定一个字符串 `s`，请你找出其中不含有重复字符的 **最长子串** 的长度。

示例 1:

```

1  输入：s = "abcabcbb"
2  输出：3
3  解释：因为无重复字符的最长子串是 "abc"，所以其长度为 3。注意 "bca" 和 "cab" 也是正确答案。

```

示例 2:

```

1  输入：s = "bbbbbb"
2  输出：1
3  解释：因为无重复字符的最长子串是 "b"，所以其长度为 1。

```

示例 3:

```

1  输入：s = "pwwkew"
2  输出：3
3  解释：因为无重复字符的最长子串是 "wke"，所以其长度为 3。
4  请注意，你的答案必须是 子串 的长度，"pwke" 是一个子序列，不是子串。

```

提示:

- `0 <= s.length <= 5 * 104`

- **s** 由英文字母、数字、符号和空格组成

解析

```
1  int lengthOfLongestSubstring(string s) {
2      // 哈希表记录每个字符上一次出现的位置
3      /*这里为什么要这么使用，是因为vector<int> last(256, -1) 是 C++ 标准库的动态
   数组，
4      它自动初始化所有元素为 -1，代码更简洁、安全。
5      题目说字符串 s 由 英文字母、数字、符号和空格 组成，这些都属于 ASCII 字符（范围
   0~127）。
6      为了安全（也能覆盖扩展 ASCII），我们直接用 0~255 共 256 个可能的字符值。
7      这样我们可以直接用字符的 ASCII 码作为下标，快速存取该字符上一次出现的位置。
8      */
9      vector<int> last(256, -1);
10
11     int left = 0;           // 滑动窗口左边界
12     int maxLen = 0;         // 最长无重复子串长度
13
14     for (int right = 0; right < s.size(); ++right) {
15         char c = s[right];
16         // 如果当前字符已经在窗口中出现过，则更新左边界
17         if (last[c] >= left) {
18             left = last[c] + 1;
19         }
20         // 记录当前字符的位置
21         last[c] = right;
22         // 更新最长长度
23         maxLen = max(maxLen, right - left + 1);
24     }
25     return maxLen;
26 }
27
```

这个算法用滑动窗口维护无重复子串，并用一个数组记录每个字符的最新位置，当遇到重复字符时，直接将窗口左边界跳到该字符上一次出现位置的下一个位置，从而在 $O(n)$ 时间内找到最长无重复子串的长度。

这里简单提一下滑动窗口相关的。

滑动窗口其实也是一种**双指针**，只是这种技巧专门解决**连续子串/子数组**的优化问题。

例如

1. **最长无重复子串**（该题）
2. **长度最小子数组**（和 $\geq$ target）
3. **固定大小窗口的最大/最小值**
4. **字符串排列 / 字母异位词**
5. **最多替换 k 个字符后最长连续串**

9.找到字符串中所有字母异位词

题目

给定两个字符串 `s` 和 `p`，找到 `s` 中所有 `p` 的 **异位词** 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。

示例 1:

```
1 输入: s = "cbaebabacd", p = "abc"
2 输出: [0,6]
3 解释:
4 起始索引等于 0 的子串是 "cba", 它是 "abc" 的异位词。
5 起始索引等于 6 的子串是 "bac", 它是 "abc" 的异位词。
```

示例 2:

```
1 输入: s = "abab", p = "ab"
2 输出: [0,1,2]
3 解释:
4 起始索引等于 0 的子串是 "ab", 它是 "ab" 的异位词。
5 起始索引等于 1 的子串是 "ba", 它是 "ab" 的异位词。
6 起始索引等于 2 的子串是 "ab", 它是 "ab" 的异位词。
```

提示:

- `1 <= s.length, p.length <= 3 * 104`
- `s` 和 `p` 仅包含小写字母

解析

其实做了上一道题，这道题就很好理解了，就是 `s` 字符串的子串要包含 `p` 字符串里面所拥有的字符种类及个数，所以说我们依然可以使用滑动窗口的方式来找子串，至于怎么对照的话，我们可以找一个东西来存储出现的字符种类和数量，这个时候，经过前面的做题，我们可能会第一时间想到哈希表，字母种类做键，数量做值，接下来我们来实现一下

```
1 vector<int> findAnagrams(string s, string p) {
2     vector<int> res;
3     int n = s.size();
4     int m = p.size();
5     if (n < m) return res;
6
7     // 哈希表: 存 p 的字符计数
8     unordered_map<char, int> need;
9     // 哈希表: 存 s 当前窗口的字符计数
10    unordered_map<char, int> window;
11
12    // 先把 p 的字符统计好
```

```

13     for (char ch : p) {
14         need[ch]++;
15     }
16
17     // 初始化第一个窗口
18     for (int i = 0; i < m; ++i) {
19         window[s[i]]++;
20     }
21
22     // 判断第一个窗口是否匹配
23     if (window == need) {
24         res.push_back(0);
25     }
26
27     // 开始滑动窗口
28     for (int i = m; i < n; ++i) {
29         // 左边字符移出窗口
30         char left = s[i - m];
31         window[left]--;
32         if (window[left] == 0) {
33             window.erase(left);
34         }
35
36         // 右边字符加入窗口
37         char right = s[i];
38         window[right]++;
39
40         // 哈希表直接对比
41         if (window == need) {
42             res.push_back(i - m + 1);
43         }
44     }
45
46     return res;
47 }

```

不止如此，我们还可以使用数组的下标映射来解决，因为题目告诉只有小写字母，所以我们可以把26个小写字母的ASCII码给映射到数组的下标去

```

1  vector<int> findAnagrams(string s, string p) {
2      vector<int> res;
3      int n = s.size();
4      int m = p.size();
5      if (n < m) return res;
6
7      // 数组下标映射: 0=a,1=b,...,25=z
8      int need[26] = {0}; // p 的字符计数
9      int window[26] = {0}; // 当前窗口的字符计数
10
11     // 初始化窗口
12     for (int i = 0; i < m; i++) {
13         need[p[i] - 'a']++; // 字符映射成下标
14         window[s[i] - 'a']++;
15     }
16

```

```

17 // 比较两个数组是否完全一样（对照）
18 if (memcmp(need, window, 26 * sizeof(int)) == 0) {
19     res.push_back(0);
20 }
21
22 // 滑动窗口
23 for (int i = m; i < n; i++) {
24     // 左边移出
25     window[s[i - m] - 'a']--;
26     // 右边移入
27     window[s[i] - 'a']++;
28
29     // 再次对照
30     if (memcmp(need, window, 26 * sizeof(int)) == 0) {
31         res.push_back(i - m + 1);
32     }
33 }
34
35 return res;
36 }

```

这两种方法在LeetCode上提交过后，我们发现使用数组的方法比使用哈希表的方法快了不少，（我的提交结果，前者48ms，后者8ms）

那么这种差异来自何处呢？

针对本题比较两者，因为本题字符限定为 26 个小写字母，我们可以利用**ASCII 码差值**，将字母映射为 0~25 的数组下标，用固定大小的数组模拟简易哈希，内存连续、下标访问无哈希计算、冲突处理与扩容开销，CPU 缓存效率极高；而 `unordered_map` 是通用哈希结构，增删查需要计算哈希值、处理冲突，且两个哈希表逐键对比的开销远大于 26 个整型数组的快速比对，因此实际提交中，数组映射解法耗时远低于哈希表解法，效率差距十分明显。

那么，还有更快的吗？

我的回答是，当然有。

```

1 vector<int> findAnagrams(string s, string p) {
2     vector<int> res;
3     int n = s.size(), m = p.size();
4     if (n < m) return res;
5
6     int cnt[26] = {0};
7     //我们知道s字符串以及其子串的顺序是不变的，所以我们可以设置一个差异数来区分是否符合要求
8     int diff = 0;
9     /*
10     这里举出一个例子，如s[c,b,a,e,a,c,b], p[a,b,c]
11     我们从最开始也就是下面说的初始窗口化，就把s里的前三个
12     给拿出来存入了数组里面，然后我们进行第一次比较，很明显
13     第一次满足，所以我们把窗口的左值存入结果数组里面。
14     */
15
16     // 统计 p 的字符
17     for (char c : p) cnt[c - 'a']++;
18
19     // 初始化窗口：把 s 前 m 个字符扣掉
20     for (int i = 0; i < m; i++) {

```

```

21     cnt[s[i] - 'a']--;
22 }
23
24 // 计算初始 diff: 有多少个字符不匹配
25 for (int i = 0; i < 26; i++) {
26     if (cnt[i] != 0) diff++;
27 }
28
29 if (diff == 0) res.push_back(0);
30 /*
31 在这里的滑动窗口就是在初始窗口的基础上向右滑动了，
32 向右滑，我们就得把左值c剔除，然后把新进来的e放进来
33 这样窗口就变成了[b,a,e]
34 */
35 // 滑动窗口
36 for (int i = m; i < n; i++) {
37     // 左边移除
38     int l = s[i - m] - 'a';
39     //这里我们要处理移出去的左值
40     //这一步是看移出去的值是不是匹配的，如果是，那就更新差异数的状态，
41     //使处于窗口不匹配状态方便下一个窗口的判断
42     if (cnt[l] == 0) diff++;
43     //这一步是更新数值，因为我们移出去了那个值，所以说现在窗口里面就差了它，
44     //就要给这个值自加，表示缺了一个
45     cnt[l]++;
46     //然后又判断，是否匹配，这里缺了就不更新差异数的状态
47     if (cnt[l] == 0) diff--;
48     //到这一步，cnt里面为[0,0,1,0,0.....],表示的c差了一个
49
50     // 右边加入
51     int r = s[i] - 'a';
52     //现在右边的e进来了，我们以同样的道理来判断
53     //cnt里面的e所对应的为0，因为不缺它
54     if (cnt[r] == 0) diff++;
55     //然后这一步就是说e是多出来的，所以这里就是cnt[e]--1
56     cnt[r]--;
57     //然后这里就是差异数变成了2
58     if (cnt[r] == 0) diff--;
59
60     //如果差异数为0，那么就说明这个窗口满足了
61     if (diff == 0) {
62         res.push_back(i - m + 1);
63     }
64     //有人会有问题，这个e带来的差异数该怎么消去呢？
65     //那么就是上面处理左值的时候解决的
66     /*
67     if (cnt[l] == 0) diff++;就说我们这个e，在这一步就是-1不满足判断对吧
68     cnt[l]++;然后在这一步他就变成0了对吧
69     if (cnt[l] == 0) diff--;这一步就是把差异数给减掉了，对吧
70     */
71 }
72
73 return res;
74 }

```

那么为啥使用diff这么快呢？

因为diff 本质是用一个变量实时维护当前窗口与目标字符串之间字符种类的差异数量，省去了普通数组解法每次滑动窗口后必须整体遍历比对 26 个字符的冗余操作，也规避了哈希表计算哈希值、处理冲突与大量键值对比对的高额开销，窗口每次滑动仅会增减两个字符，算法只会局部更新这两个字符的匹配状态，只在字符由匹配变为不匹配时让 diff 自增、由不匹配恢复为匹配时让 diff 自减，全程没有任何批量遍历和多余比对，最终仅通过判断 diff 是否为 0 就能直接确定窗口是否为异位词，极致压缩了常数时间开销，因此运行速度远快于另外两种写法。

10.和为K的子数组

题目

给你一个整数数组 `nums` 和一个整数 `k`，请你统计并返回 该数组中和为 `k` 的子数组的个数。

子数组是数组中元素的连续非空序列。

示例 1：

```
1 输入：nums = [1,1,1]，k = 2
2 输出：2
```

示例 2：

```
1 输入：nums = [1,2,3]，k = 3
2 输出：2
```

提示：

- `1 <= nums.length <= 2 * 104`
- `-1000 <= nums[i] <= 1000`
- `-107 <= k <= 107`

解析

其实做了前面的题目就能发现，这种找一段连续元素、满足和为某个值的题目，一般都可以通过差值来匹配是否满足，核心思路就是用前缀和算出区间差值，再用哈希表快速查找，效率非常高。

```
1 int subarraySum(vector<int>& nums, int k) {
2     unordered_map<int, int> map;
3     // 初始化：前缀和 0 出现 1 次
4     map[0] = 1;
5
6     int preSum = 0; // 前缀和
7     int count = 0; // 满足条件的子数组数量
8
9     for (int num : nums) {
10         preSum += num; // 更新当前前缀和
11
12         // 查找前面有多少个前缀和 = preSum - k
```

```

13         if (map.find(preSum - k) != map.end()) {
14             count += map[preSum - k];
15         }
16
17         // 把当前前缀和存入哈希表
18         map[preSum]++;
19     }
20
21     return count;
22 }

```

这里我们要讲明白子数组是啥，子数组就是**数组中一段连续、不可分割的元素序列**，绝对不能跳跃、不能挑元素、不能断开。

所以说这个就很简单了，也很好理解我们这道题为什么要用前缀和了。

例如[1,2,3,2,1]，满足k=3

前缀和就是前面每一个加起来

第 1 个元素前缀和：只加第一个数 → 1

第 2 个元素前缀和：第一个数 + 第二个数 → 1+2=3

第 3 个元素前缀和：前三个数相加 → 1+2+3=6

第 4 个元素前缀和：前四个数相加 → 1+2+3+2=8

第 5 个元素前缀和：前五个数相加 → 1+2+3+2+1=9

我们可以直观的看出来满足的是[1,2],[2,1]这两个对吧

第一个我们可以怎么得到呢？

是不是就是第二个元素的前缀和。

第二个我们怎么得到？

是不是第五个元素和第三个元素的前缀和相减得到。

前缀和和目标值的差值如果不存在哈希表里面，就存进去，如果存在，就说明能够通过前面的前缀和减出来。

满足的计数，这样就把知道题目解决了，使用的就是前缀和和哈希表共同解决的

11.滑动窗口最大值

题目

给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。

返回 *滑动窗口中的最大值*。

示例 1：

```

1  输入: nums = [1,3,-1,-3,5,3,6,7], k = 3
2  输出: [3,3,5,5,6,7]
3  解释:
4  滑动窗口的位置          最大值
5  -----
6  [1  3  -1] -3  5  3  6  7      3
7  1 [3  -1  -3] 5  3  6  7      3
8  1  3 [-1  -3  5] 3  6  7      5
9  1  3 -1 [-3  5  3] 6  7      5
10 1  3 -1 -3 [5  3  6] 7      6
11 1  3 -1 -3  5 [3  6  7]      7

```

示例 2:

```

1  输入: nums = [1], k = 1
2  输出: [1]

```

提示:

- `1 <= nums.length <= 105`
- `-104 <= nums[i] <= 104`
- `1 <= k <= nums.length`

解析

看到这道题时的第一思路也很简单啊，就是对窗口内的数字进行大小比较即可，我们首先来实现一下这个思路

```

1  vector<int> maxSlidingWindow(vector<int>& nums, int k) {
2      vector<int> res;
3      int l = 0;
4      int r = k - 1;
5
6      while(r < nums.size()){
7          int max = nums[l];
8          for(int i = l + 1; i <= r; i++){
9              if(nums[i] > max){
10                 max = nums[i];
11             }
12         }
13         res.push_back(max);
14         l++;
15         r++;
16     }
17     return res;
18 }

```

尽管这样确实能实现，但是时间花的就很多了，时间复杂度为 $O(n*k)$ ，提交LeetCode也是会报超时错误。

所以我们应该怎么来优化呢？

我们可以使用双端队列，也叫做单调队列来完成

```
1  vector<int> maxSlidingWindow(vector<int>& nums, int k) {
2      vector<int> res;
3      // 双端队列：存储元素下标，对应数值单调递减
4      deque<int> q;
5
6      for (int i = 0; i < nums.size(); ++i) {
7          // 1. 移除队列中 超出窗口左边界 的元素
8          while (!q.empty() && q.front() < i - k + 1) {
9              q.pop_front();
10         }
11
12         // 2. 维护单调递减：移除队列尾部比当前元素小的值
13         while (!q.empty() && nums[q.back()] < nums[i]) {
14             q.pop_back();
15         }
16
17         // 3. 当前元素下标入队
18         q.push_back(i);
19
20         // 4. 窗口形成后，记录队首（最大值）
21         if (i >= k - 1) {
22             res.push_back(nums[q.front()]);
23         }
24     }
25     return res;
26 }
```

其实这里除了使用STL里面的deque，还可以自己写个静态数组来模拟

```
1  vector<int> maxSlidingWindow(vector<int>& nums, int k) {
2      int n = nums.size(); // 数组长度
3      int head = 0, tail = 0; // 队头、队尾指针
4      int q[n]; // 用数组模拟队列
5      vector<int> ans; // 存答案
6
7      for(int i = 0; i < n; i++){ // i 是窗口右边界
8
9          // 1. 如果队头已经滑出窗口 → 删掉
10         if(q[head] == i - k) head++;
11
12         // 2. 队尾比当前数小 → 全部删掉（保持递减）
13         while(head < tail && nums[q[tail - 1]] <= nums[i])
14             tail--;
15
16         // 3. 把当前下标加入队尾
17         q[tail++] = i;
18
19         // 4. 窗口形成 → 队头就是最大值
20         if(i >= k - 1)
21             ans.push_back(nums[q[head]]);
22     }
```

```
23     return ans;
24 }
```

这样又能快一些，因为这里使用静态数组是用的连续的存储空间，而使用deque是链表，是不连续的存储空间

那么还有能够解决的方式吗？

有的，兄弟。

```
1  vector<int> maxSlidingWindow(vector<int>& nums, int k) {
2      int n = nums.size();
3      vector<int> prefixMax(n), suffixMax(n);
4
5      //求前缀最大值（块内从左到右）
6      for (int i = 0; i < n; ++i) {
7          if (i % k == 0) {
8              prefixMax[i] = nums[i];    // 新块开始
9          } else {
10             prefixMax[i] = max(prefixMax[i-1], nums[i]);
11         }
12     }
13
14     // 求后缀最大值（块内从右到左）
15     for (int i = n-1; i >= 0; --i) {
16         if (i == n-1 || (i+1) % k == 0) {
17             suffixMax[i] = nums[i];    // 新块结束
18         } else {
19             suffixMax[i] = max(suffixMax[i+1], nums[i]);
20         }
21     }
22
23     //计算答案
24     vector<int> ans;
25     for (int i = 0; i <= n-k; ++i) {
26         ans.push_back( max( suffixMax[i], prefixMax[i+k-1] ) );
27     }
28
29     return ans;
30 }
```

这种滑动窗口，我们很容易就知道这个窗口最多也就跨了两个块，这个块就是一个窗口的大小，那么就可以仅针对于这两个块来找出最大值就可以了，

上面代码所属前缀最大值和后缀最大值的意思也很好理解，一个值从前往后找出最大的，一个则是从后往前，可为什么要这样呢，举个例子就好理解了

下标: 0 1 2 | 3 4 5 | 6 7

数值: 1 3 -1 |-3 5 3 | 6 7

k=3，我们在这里举例的时候先把它分一下块，

计算前缀最大值数组

i=0 → 0%3==0 → prefixMax[0]=1

$i=1 \rightarrow \max(1,3)=3$

$i=2 \rightarrow \max(3,-1)=3$

$i=3 \rightarrow 3\%3==0 \rightarrow -3$

$i=4 \rightarrow \max(-3,5)=5$

$i=5 \rightarrow \max(5,3)=5$

$i=6 \rightarrow 6\%3==0 \rightarrow 6$

$i=7 \rightarrow \max(6,7)=7$

就得到了一个对应下标下的前缀最大值数组

`prefixMax = [1,3,3, -3,5,5, 6,7]`

同样的道理倒过来计算，就有了后缀最大值数组

`suffixMax = [3,3,-1, 5,5,3, 7,7]`

有了这两个之后，我们来看一下第一个窗口，也就是`[1,3,-1]`

我们一眼就能看出来是3

从两个数组来看呢，比较窗口开头下标对应的后缀最大值数组的对应值和窗口末尾下标对应的前缀最大值数组的对应值。

我们发现是不是比较结果也是3

这是因为前缀和后缀正好在窗口内做了个交叉，把最大的部分放在了各自的开头和末尾，这样就正好比较这两数就可以判断出来这一个窗口里的最大值了

12.最小覆盖子串

题目

给定两个字符串 `s` 和 `t`，长度分别是 `m` 和 `n`，返回 `s` 中的 **最短窗口子串**，使得该子串包含 `t` 中的每一个字符（包括重复字符）。如果没有这样的子串，返回空字符串 `""`。

测试用例保证答案唯一。

示例 1:

- 1 输入: `s = "ADOBECODEBANC"`, `t = "ABC"`
- 2 输出: `"BANC"`
- 3 解释: 最小覆盖子串 `"BANC"` 包含来自字符串 `t` 的 `'A'`、`'B'` 和 `'C'`。

示例 2:

- 1 输入: `s = "a"`, `t = "a"`
- 2 输出: `"a"`
- 3 解释: 整个字符串 `s` 是最小覆盖子串。

示例 3:

```
1 输入: s = "a", t = "aa"
2 输出: ""
3 解释: t 中两个字符 'a' 均应包含在 s 的子串中,
4 因此没有符合条件的子字符串, 返回空字符串。
```

提示:

- `m == s.length`
- `n == t.length`
- `1 <= m, n <= 105`
- `s` 和 `t` 由英文字母组成

进阶: 你能设计一个在 $O(m + n)$ 时间内解决此问题的算法吗?

解析

经过前面题的学习, 我们对于找子串的思路也就是滑动窗口加上哈希计数

所以有以下代码实现

```
1  string minWindow(string s, string t) {
2      // 题目提到只有英文字母组成, 所以说我们依然可以采用转换为ASCII码来计数
3      vector<int> need(128, 0);
4      vector<int> window(128, 0);
5
6      //统计t需要的字符数量
7      for (char c : t) need[c]++;
8
9      int left = 0, right = 0;
10     int count = 0;          // 满足数量的字符种类
11     int needKind = 0;       // t一共有多少种不同字符
12     int start = 0, len = 1e9; // 记录最优子串起点, 长度找个大的值方便后续进行第一
    次比较
13
14     // 统计t的字符种类
15     for (int i = 0; i < 128; i++)
16         if (need[i] > 0) needKind++;
17
18     while (right < s.size()) {
19         char c = s[right];
20         window[c]++;
21
22         // 当前字符数量刚好达标 -> 合法种类+1
23         if (need[c] != 0 && window[c] == need[c])
24             count++;
25
26         // 窗口合法: 全部字符都满足, 开始收缩左边界
27         while (count == needKind) {
28             // 更新最小子串
29             if (right - left + 1 < len) {
30                 len = right - left + 1;
```

```

31         start = left;
32     }
33
34     // 左指针右移，移出字符
35     char d = s[left];
36     if (need[d] != 0 && window[d] == need[d])
37         count--; // 不满足了，合法种类-1
38     window[d]--;
39     left++;
40 }
41 right++;
42 }
43 return len == 1e9 ? "" : s.substr(start, len);
44 }

```

13.最大子数组和

题目

给你一个整数数组 `nums`，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

子数组是数组中的一个连续部分。

示例 1:

```

1 输入: nums = [-2,1,-3,4,-1,2,1,-5,4]
2 输出: 6
3 解释: 连续子数组 [4,-1,2,1] 的和最大，为 6 。

```

示例 2:

```

1 输入: nums = [1]
2 输出: 1

```

示例 3:

```

1 输入: nums = [5,4,-1,7,8]
2 输出: 23

```

提示:

- `1 <= nums.length <= 105`
- `-104 <= nums[i] <= 104`

进阶: 如果你已经实现复杂度为 `O(n)` 的解法，尝试使用更为精妙的 **分治法** 求解。

解析

这道题有个经典的算法来解决它，就是kadane算法。

Kadane 算法，就是数学家 **Joseph Kadane**，专门为**最大连续子数组**和这个经典问题，量身设计的线性解法。

下面是代码实现

```
1  int maxSubArray(vector<int>& nums) {
2      int cur = nums[0]; //以当前这个数为结尾的最大连续子数组和
3      int res = nums[0]; //全局历史最大值，全程记录整段数组里答案
4      for(int i = 1; i < nums.size(); ++i)
5          {
6              cur = max(nums[i], cur + nums[i]); //比较以这个数字为开头和连接这个数字
              带来的最大和谁更大，这是一种贪心的思想
7              res = max(res, cur);
8          }
9      return res;
10 }
```

我们依然可以用分治法来解决这个问题

```
1  int maxSubArray(vector<int>& nums) {
2      // 调用分治函数，范围：整个数组 0 ~ nums.size()-1
3      return divide(nums, 0, nums.size() - 1);
4  }
5
6  // 分治核心函数
7  // 求 nums[l ... r] 区间内的最大子数组和
8  int divide(vector<int>& nums, int l, int r) {
9      // 递归出口：区间只有一个数，最大和就是它自己
10     if (l == r) {
11         return nums[l];
12     }
13
14     //找中间点，把数组切成两半
15     int mid = (l + r) / 2;
16
17     //递归求三部分最大值
18     int leftMax = divide(nums, l, mid); // 完全在左边
19     int rightMax = divide(nums, mid + 1, r); // 完全在右边
20     int crossMax = getCrossMax(nums, l, mid, r); // 横跨中间
21
22     //三个里面最大的就是答案
23     return max( max(leftMax, rightMax), crossMax );
24 }
25
26 //最关键：求【横跨 mid】的最大子数组和
27 //这种子数组一定包含 nums[mid] 和 nums[mid+1]
28 int getCrossMax(vector<int>& nums, int l, int mid, int r) {
29
30     // 第一步：从 mid 向左走，找最大后缀和
31     int leftSum = INT_MIN; // 记录左边最大和
32     int curSum = 0; // 临时累加
```

```

33
34     for (int i = mid; i >= l; i--) {
35         curSum += nums[i];           // 不断往左加
36         leftSum = max(leftSum, curSum); // 更新左边最大
37     }
38
39     // 第二步: 从 mid+1 向右走, 找最大前缀和
40     int rightSum = INT_MIN;
41     curSum = 0;
42
43     for (int i = mid + 1; i <= r; i++) {
44         curSum += nums[i];           // 不断往右加
45         rightSum = max(rightSum, curSum); // 更新右边最大
46     }
47     // 横跨中间的最大和 = 左边最大 + 右边最大
48     return leftSum + rightSum;
49 }

```

14.合并区间

题目

以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [starti, endi]`。请你合并所有重叠的区间，并返回 一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间。

示例 1:

```

1  输入: intervals = [[1,3],[2,6],[8,10],[15,18]]
2  输出: [[1,6],[8,10],[15,18]]
3  解释: 区间 [1,3] 和 [2,6] 重叠，将它们合并为 [1,6]。

```

示例 2:

```

1  输入: intervals = [[1,4],[4,5]]
2  输出: [[1,5]]
3  解释: 区间 [1,4] 和 [4,5] 可被视为重叠区间。

```

示例 3:

```

1  输入: intervals = [[4,7],[1,4]]
2  输出: [[1,7]]
3  解释: 区间 [1,4] 和 [4,7] 可被视为重叠区间。

```

提示:

- `1 <= intervals.length <= 104`

- `intervals[i].length == 2`
- `0 <= starti <= endi <= 104`

解析

这道题也很好理解，本质就是依靠区间端点的数值大小进行判断，先将所有区间按照起始位置升序排序，让重叠区间彼此相邻，再依次遍历，比较当前区间起点与结果集中最后一个区间的终点，若当前起点小于等于该终点，说明区间重叠或连续，就合并更新最大右端点，若不满足则直接存入新区间，全程仅通过端点比较，即可完成所有区间的合并，最终输出无重叠的完整区间

```

1  vector<vector<int>> merge(vector<vector<int>>& intervals) {
2      // 1. 先按区间【起点】从小到大排序（必须！）
3      sort(intervals.begin(), intervals.end());
4
5      vector<vector<int>> res; // 存最终答案
6
7      // 2. 遍历每一个区间
8      for(auto& interval : intervals) {
9          int start = interval[0]; // 当前区间起点
10         int end = interval[1];    // 当前区间终点
11
12         // 如果结果为空，直接放进去
13         if(res.empty()) {
14             res.push_back(interval);
15         }
16         else {
17             // 拿到结果里【最后一个区间】
18             auto& last = res.back();
19             int lastEnd = last[1];
20
21             // 情况1: 重叠/相连 → 合并
22             if(start <= lastEnd) {
23                 // 更新最后一个区间的结尾为最大值
24                 last[1] = max(lastEnd, end);
25             }
26             // 情况2: 不重叠 → 直接加入
27             else {
28                 res.push_back(interval);
29             }
30         }
31     }
32     return res;
33 }
```

15.轮转数组

题目

给定一个整数数组 `nums`，将数组中的元素向右轮转 `k` 个位置，其中 `k` 是非负数。

示例 1:

```
1  输入: nums = [1,2,3,4,5,6,7], k = 3
2  输出: [5,6,7,1,2,3,4]
3  解释:
4  向右轮转 1 步: [7,1,2,3,4,5,6]
5  向右轮转 2 步: [6,7,1,2,3,4,5]
6  向右轮转 3 步: [5,6,7,1,2,3,4]
```

示例 2:

```
1  输入: nums = [-1,-100,3,99], k = 2
2  输出: [3,99,-1,-100]
3  解释:
4  向右轮转 1 步: [99,-1,-100,3]
5  向右轮转 2 步: [3,99,-1,-100]
```

提示:

- `1 <= nums.length <= 105`
- `-231 <= nums[i] <= 231 - 1`
- `0 <= k <= 105`

进阶:

- 尽可能想出更多的解决方案, 至少有 **三种** 不同的方法可以解决这个问题。
- 你可以使用空间复杂度为 `O(1)` 的 **原地** 算法解决这个问题吗?

解析

这道题我们的第一思路就是很简单的使用暴力的方法来求解, 思路为每次把最后一个元素挪到最前面, 重复 k 次。

```
1  void rotate(vector<int>& nums, int k) {
2      int n = nums.size();
3      k = k % n;
4
5      for (int i = 0; i < k; i++) {
6          // 取出最后一个元素
7          int last = nums.back();
8          // 从后往前覆盖
9          for (int j = n - 1; j > 0; j--) {
10             nums[j] = nums[j - 1];
11         }
12         // 放到第一位
13         nums[0] = last;
14     }
15 }
```

但我们很容易就知道这个算法的时间复杂度: $O(k*n)$ (两层循环, 数据大时超时) 空间复杂度: $O(1)$ (原地操作), 节约了空间但是放大了时间, 得不偿失

第二种方法就是创建一个临时数组直接来存放移位后的值，然后再赋值回去

```
1 void rotate(vector<int>& nums, int k) {
2     int n = nums.size();
3     k = k % n;
4
5     // 新建临时数组
6     vector<int> temp(n);
7
8     // 公式: 原位置 i → 新位置 (i + k) % n
9     for (int i = 0; i < n; i++) {
10         temp[(i + k) % n] = nums[i];
11     }
12
13     // 复制回原数组
14     nums = temp;
15 }
```

它的时间复杂度: $O(n)$ (一次遍历) 空间复杂度: $O(n)$ (开了新数组), 整体思路相当好理解

第三种方法使用的叫一种三次翻转法

```
1 void rotate(vector<int>& nums, int k) {
2     int n = nums.size();
3     k = k % n;
4
5     // 三次反转
6     reverse(nums.begin(), nums.end());           // 1. 全反转
7     reverse(nums.begin(), nums.begin() + k);     // 2. 反转前 k 个
8     reverse(nums.begin() + k, nums.end());       // 3. 反转后 n-k 个
9 }
```

时间复杂度: $O(n)$ 空间复杂度: $O(1)$ (原地操作)

第四种方法叫做环状替换法

思路为, 数组旋转可以看成**环形移动**: 每个元素直接跳到它最终的位置, 被挤走的元素继续跳, 直到回到起点。用 `count` 记录移动了多少个元素, 全部移动完就结束。

```
1 void rotate(vector<int>& nums, int k) {
2     int n = nums.size();
3     k = k % n;
4     int count = 0; // 记录已经移动的元素个数
5
6     // 直到所有元素都移动完成
7     for (int start = 0; count < n; start++) {
8         int cur = start;           // 当前位置
9         int prev = nums[cur];      // 当前要保存的值
10
11        // 环状循环, 直到回到起点
12        do {
13            // 计算下一个位置
14            int nextIdx = (cur + k) % n;
15
16            // 保存下一个位置的值, 把当前值放进去
```

```

17         int temp = nums[nextIdx];
18         nums[nextIdx] = prev;
19
20         // 更新：下一个位置变成当前，被替换的值变成 prev
21         prev = temp;
22         cur = nextIdx;
23         count++;
24     } while (start != cur); // 回到起点就结束这一环
25 }
26 }

```

时间复杂度： $O(n)$ （每个元素只移动一次）空间复杂度： $O(1)$

这个看起来就没有那么好理解了

其核心本质就是把数组看成一个圆环，每个元素最终要去的位置：

$next = (cur + k) \bmod n$

从某个起点出发，把元素挨个往后挪，绕环走一圈，回到起点就结束这一圈；

换下个起点，继续循环，直到**所有 n 个元素都就位**。

说直白点就是第一种方法暴力法的进阶版本，只是让每一个移动的数不用挨着挨着移到后面去，而是直接移动到目标位置。

这个所谓的环就是在移动 k 次下，下标以 k 次为间隔可以形成一个环，处于这些下标的值永远都是那么几个值，就是这几个值在环里面转圈圈

例如一个容量为5的数组， $[0,1,2,3,4]$ 这里面是下标不是值，

当 $k=2$ 时

$0 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow 3 \rightarrow 0$

我们发现了什么？

是不是这个环正好把这个数组给串上了

其实还有多个环的情况

如容量为6的数组， $k=2$

环 1: $0 \rightarrow 2 \rightarrow 4 \rightarrow 0$

环 2: $1 \rightarrow 3 \rightarrow 5 \rightarrow 1$

处理完每个环，就相当于处理完了整个数组了

16.除了自身以外数组的乘积

题目

给你一个整数数组 `nums`，返回 数组 `answer`，其中 `answer[i]` 等于 `nums` 中除了 `nums[i]` 之外其余各元素的乘积。

题目数据 **保证** 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 **32 位** 整数范围内。

请 **不要使用除法**，且在 $O(n)$ 时间复杂度内完成此题。

示例 1:

```
1 输入: nums = [1,2,3,4]
2 输出: [24,12,8,6]
```

示例 2:

```
1 输入: nums = [-1,1,0,-3,3]
2 输出: [0,0,9,0,0]
```

提示:

- `2 <= nums.length <= 105`
- `-30 <= nums[i] <= 30`
- 输入 **保证** 数组 `answer[i]` 在 **32 位** 整数范围内

进阶: 你可以在 `O(1)` 的额外空间复杂度内完成这个题目吗? (出于对空间复杂度分析的目的, 输出数组 **不被视为** 额外空间。)

解析

还是老办法啊, 直接暴力法先来一遍

```
1 vector<int> productExceptSelf(vector<int>& nums) {
2     vector<int> answer;
3
4     int n = nums.size();
5     for(int i = 0; i < n; i++){
6         int b = 1;
7         for(int j = 0; j < n; j++){
8             if(i != j){
9                 b = b*nums[j];
10            }
11        }
12        answer.push_back(b);
13    }
14    return answer;
15 }
```

一样的问题, 能解决但是会超时

所以说我们继续优化

这种方法通过左右乘积来进行计算

```
1 vector<int> productExceptSelf(vector<int>& nums) {
2     int n = nums.size();
3     vector<int> res(n, 1);
4     int left = 1;    // 这是个累乘盒子, 从左往右攒乘积
5 }
```

```

6      // 第一趟：从左往右
7      for (int i = 0; i < n; i++) {
8          res[i] = left;      // 先把盒子里已有的值给当前位置
9          left *= nums[i];    // 再把当前数装进盒子，给后面用
10     }
11
12     int right = 1; // 另一个累乘盒子，从右往左攒乘积
13     // 第二趟：从右往左
14     for (int i = n-1; i >= 0; i--) {
15         res[i] *= right;    // 先把右边盒子已有值乘给当前位置
16         right *= nums[i];   // 再把当前数装进盒子，给前面用
17     }
18     return res;
19 }

```

举例说明，[2,3,4,5,6]

从左往右，left初值为1，直接赋值给结果数组，我们很清晰的能看到并没有乘上自身，然后left乘上这个下标的数给到下个下标的结果数组使用，说直白点，就是通过left来累乘除了自身下标下的数，然后放到自身下标的结果数组里这样我们就得到了没有我本身的左边乘积总值，那么右边咋办呢，在从右往左来一遍，道理是一样的，这左右值一乘起来不就是除了自身值的乘积总值么。

而且这从左往右，从右往左，只把数组遍历了一遍，进而我们就明白这个的时间复杂度为 $O(n)$

时间大大的减小了。

17.缺失的第一个正数

题目

给你一个未排序的整数数组 `nums`，请你找出其中没有出现的最小的正整数。

请你实现时间复杂度为 $O(n)$ 并且只使用常数级别额外空间的解决方案。

示例 1:

```

1  输入: nums = [1,2,0]
2  输出: 3
3  解释: 范围 [1,2] 中的数字都在数组中。

```

示例 2:

```

1  输入: nums = [3,4,-1,1]
2  输出: 2
3  解释: 1 在数组中，但 2 没有。

```

示例 3:

```

1  输入: nums = [7,8,9,11,12]
2  输出: 1
3  解释: 最小的正数 1 没有出现。

```

提示：

- `1 <= nums.length <= 105`
- `-231 <= nums[i] <= 231 - 1`

解析

给的是一个没有排序的整数数组，找的是没有出现的最小正整数，所以说我们要找的目标的大小肯定是大于0的，这样子就很好解决了。

我们要解答这个题，这个没有排序显然会增加难度，所以说我们要进行排序，但是不是全部都来一遍排序，我们只需要管目标值可能出现的地方就可以了，也就是正整数部分。

这里也有一个技巧，也就是利用数组的下标来表示实际的值，具体的，我在代码里面说明

```
1  int firstMissingPositive(vector<int>& nums) {
2      int n = nums.size();
3      //我们可以直接利用本身提供的数组来进行这个所谓的标记操作
4      //具体就是遍历每个位置，把数字按顺序摆放起来
5      //我这里就是按照下标对应的值是下标+1，例如nums[0]=1,这么来放置
6      for (int i = 0; i < n; ++i) {
7          // 只要这个数是 1~n 之间，并且没放对位置，就一直交换
8          while (nums[i] > 0 && nums[i] <= n && nums[nums[i]-1] != nums[i]) {
9              swap(nums[nums[i]-1], nums[i]);
10         }
11     }
12
13     //然后再遍历一遍，找第一个不对的位置，也就是我们要找的那个值了
14     for (int i = 0; i < n; ++i) {
15         if (nums[i] != i + 1) {
16             return i + 1;
17         }
18     }
19
20     // 如果1~n 都齐了，返回 n+1
21     return n + 1;
22 }
```

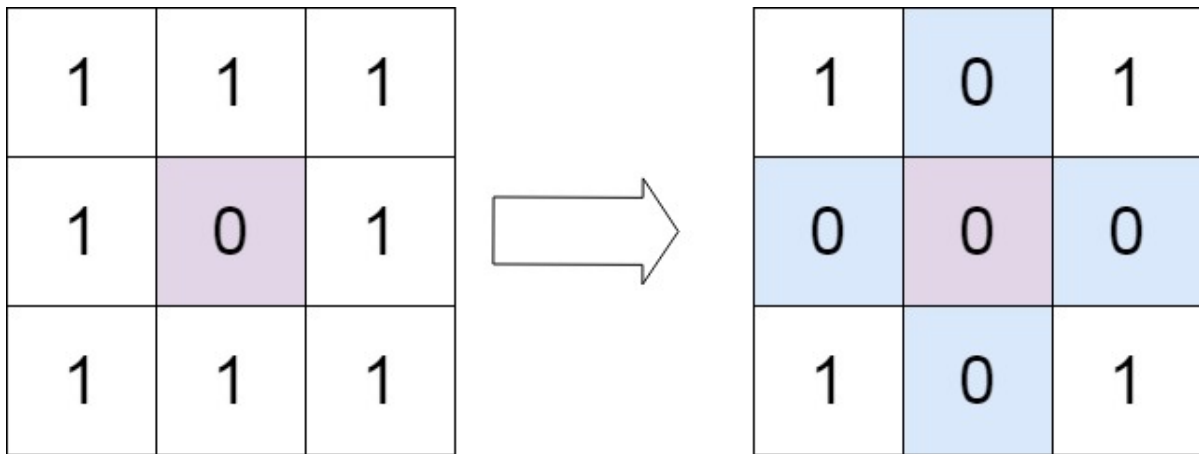
这里也就是利用原数组来做标记，同样我们也可以用一个额外的数组来做这个标记，只是我们为了节约内存空间才这么做的

18.矩阵置零

题目

给定一个 `m * n` 的矩阵，如果一个元素为 `0`，则将其所在行和列的所有元素都设为 `0`。请使用 [原地](#) 算法。

示例 1：

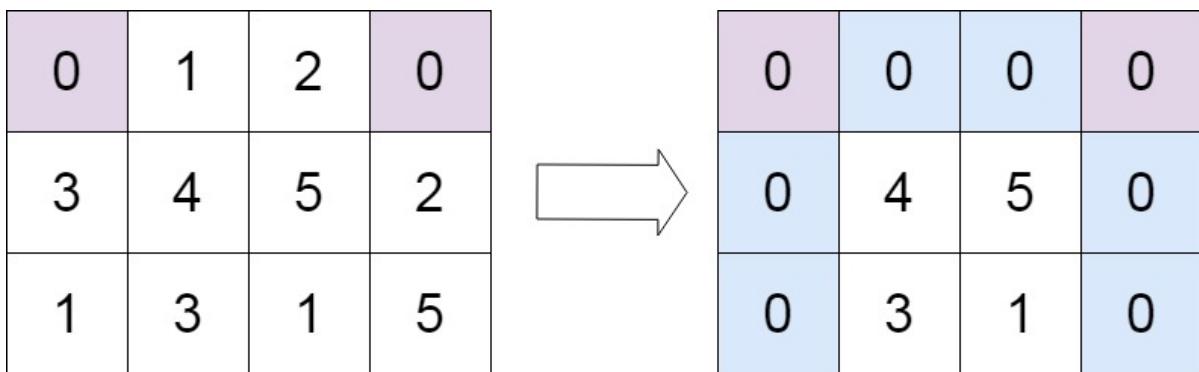


```

1  输入: matrix = [[1,1,1],[1,0,1],[1,1,1]]
2  输出: [[1,0,1],[0,0,0],[1,0,1]]

```

示例 2:



```

1  输入: matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]
2  输出: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]

```

提示:

- `m == matrix.length`
- `n == matrix[0].length`
- `1 <= m, n <= 200`
- `-231 <= matrix[i][j] <= 231 - 1`

进阶:

- 一个直观的解决方案是使用 $O(m*n)$ 的额外空间, 但这并不是一个好的解决方案。
- 一个简单的改进方案是使用 $O(m + n)$ 的额外空间, 但这仍然不是最好的解决方案。
- 你能想出一个仅使用常量空间的解决方案吗?

解析

呃，这道题我们看进阶里面的提示应该是有三种方法，我们先来说第一种

```
1 void setZeroes(vector<vector<int>>& matrix) {
2     int m = matrix.size();
3     int n = matrix[0].size();
4     //这个就是使用额外的一个矩阵来存放原来的矩阵，通过这个新的矩阵来作为寻找0的地图
5     //因为我们直接在原数组上置0的话，那么就很容易忽略其它的本来存在的0
6     vector<vector<int>> temp = matrix;
7
8     for (int i = 0; i < m; i++) {
9         for (int j = 0; j < n; j++) {
10             if (temp[i][j] == 0) {
11                 //这里找到了，我们就分别置0
12                 // 置0第 i 行
13                 for (int k = 0; k < n; k++) matrix[i][k] = 0;
14                 // 置0第 j 列
15                 for (int k = 0; k < m; k++) matrix[k][j] = 0;
16             }
17         }
18     }
19 }
```

这个就是空间复杂度为 $O(mn)$ 的解法，因为使用了一个和原来一模一样的新矩阵。

```
1 void setZeroes(vector<vector<int>>& matrix) {
2     int m = matrix.size();
3     int n = matrix[0].size();
4     //这个的话，就是创建两个新的数组，一个来放行，一个来放列那些行列存在0
5     //为什么要这么做，因为，置0要求是该行该列都置0，所以说如果行或列里面存在多个0
6     //其实都可以等价为一个0来处理
7     vector<bool> row(m, false);
8     vector<bool> col(n, false);
9
10    // 记录哪些行、列有 0
11    for (int i = 0; i < m; i++) {
12        for (int j = 0; j < n; j++) {
13            if (matrix[i][j] == 0) {
14                row[i] = true;
15                col[j] = true;
16            }
17        }
18    }
19
20    // 置0行
21    for (int i = 0; i < m; i++) {
22        if (row[i]) {
23            for (int j = 0; j < n; j++) matrix[i][j] = 0;
24        }
25    }
26
27    // 置0列
28    for (int j = 0; j < n; j++) {
```

```

29         if (col[j]) {
30             for (int i = 0; i < m; i++) matrix[i][j] = 0;
31         }
32     }
33 }

```

这就是第二种方法，也就是空间复杂度为 $O(m+n)$ 的解法，使用了两个数组来做标记

```

1 void setZeroes(vector<vector<int>>& matrix) {
2     int m = matrix.size();
3     int n = matrix[0].size();
4     //方法三其实是在方法二上的升级版，也就是它没有使用额外的数组来做标记，而是使用矩阵本身
    来做标记
5     //也就是充分利用利用置0是包括列和行的，那么如果该行或该列存在0
6     //那么我们就可以使用矩阵的第一行和第一列来存储标记
7     bool row0 = false, col0 = false;
8
9     // 这里记录第一行、第一列原本有没有 0，如果有的话，在最后要对其进行置0操作
10    for (int j = 0; j < n; j++) if (matrix[0][j] == 0) row0 = true;
11    for (int i = 0; i < m; i++) if (matrix[i][0] == 0) col0 = true;
12
13    // 用第一行、第一列做标记
14    for (int i = 1; i < m; i++) {
15        for (int j = 1; j < n; j++) {
16            if (matrix[i][j] == 0) {
17                matrix[i][0] = 0;
18                matrix[0][j] = 0;
19            }
20        }
21    }
22
23    // 上面遍历矩阵把应该置0的行列做了标记，之后就根据标记置0
24    for (int i = 1; i < m; i++) {
25        if (matrix[i][0] == 0) {
26            for (int j = 1; j < n; j++) matrix[i][j] = 0;
27        }
28    }
29    for (int j = 1; j < n; j++) {
30        if (matrix[0][j] == 0) {
31            for (int i = 1; i < m; i++) matrix[i][j] = 0;
32        }
33    }
34
35    // 最后根据原本是否存在0来补上处理第一行、第一列的置0操作
36    if (row0) for (int j = 0; j < n; j++) matrix[0][j] = 0;
37    if (col0) for (int i = 0; i < m; i++) matrix[i][0] = 0;
38 }

```

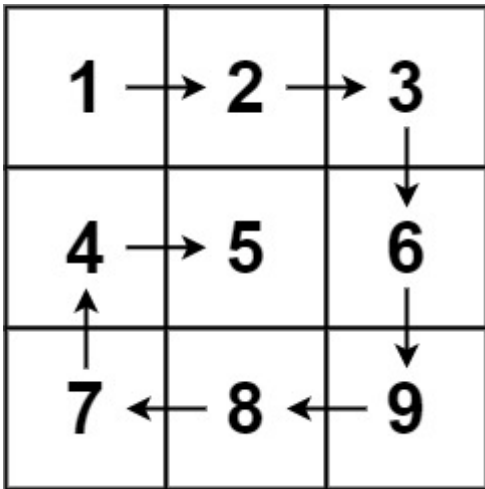
这也就是空间复杂度为 $O(1)$ 的解法，自己用自己来做标记存储

19.螺旋矩阵

题目

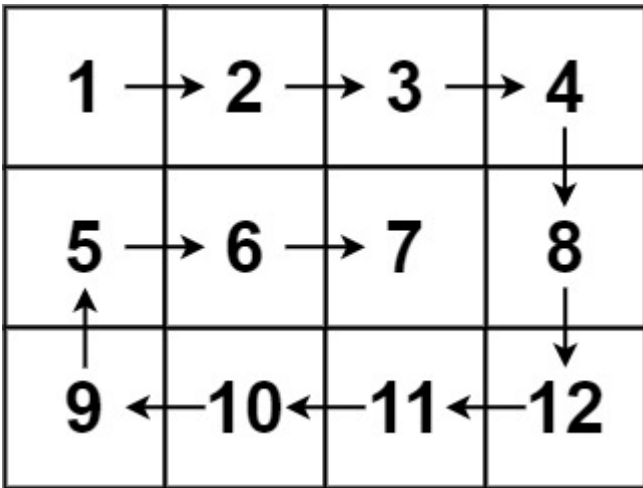
给你一个 `m` 行 `n` 列的矩阵 `matrix`，请按照 **顺时针螺旋顺序**，返回矩阵中的所有元素。

示例 1:



```
1 输入: matrix = [[1,2,3],[4,5,6],[7,8,9]]
2 输出: [1,2,3,6,9,8,7,4,5]
```

示例 2:



```
1 输入: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
2 输出: [1,2,3,4,8,12,11,10,9,5,6,7]
```

提示:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 10`
- `-100 <= matrix[i][j] <= 100`

解析

这道题也很好理解和书写啊，就是绕着输出一遍就是了，我们就循着这个思路来写

```
1  vector<int> spiralOrder(vector<vector<int>>& matrix) {
2      //首先，我们先拿到目标矩阵的长和宽
3      int m = matrix.size();
4      int n = matrix[0].size();
5
6      vector<int> result;
7      //接下来我们来转圈
8      //外层的for循环就是我们一共要转几圈，里面的for是来针对这个圈来输出的
9      //这里的循环条件的(min(m, n) + 1) / 2，就是在计算圈数
10     //取min是因为短边决定圈数
11     //加1是为了向上取整，避免漏圈
12     //除以2是因为每绕一圈，矩阵的短边长度会减少2，所以绕圈数就是短边长度减半（向上取
    整）。
13     for(int i = 0; i < (min(m, n) + 1) / 2; i++){
14         //这里是在绕圈的上边
15         for(int j = i; j < n - i; j++){
16             result.push_back(matrix[i][j]);
17         }
18         //这里是在绕圈的右边
19         for(int j = i + 1; j < m - i; j++){
20             result.push_back(matrix[j][n - i - 1]);
21         }
22         //下面的两个if都是用来防止重复的
23         if(m - i - 1 != i){
24             //绕下边
25             for(int j = n - i - 2; j >= i; j--){
26                 result.push_back(matrix[m - i - 1][j]);
27             }
28         }
29         if(n - i - 1 != i){
30             //绕左边
31             for(int j = m - i - 2; j > i; j--){
32                 result.push_back(matrix[j][i]);
33             }
34         }
35     }
36     return result;
37 }
```

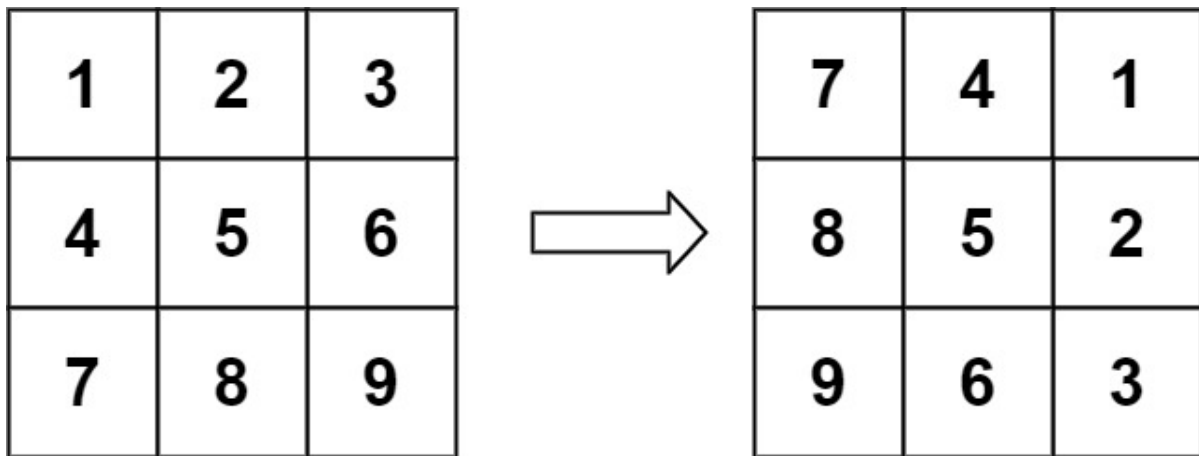
20.旋转图像

题目

给定一个 $n \times n$ 的二维矩阵 `matrix` 表示一个图像。请你将图像顺时针旋转 90 度。

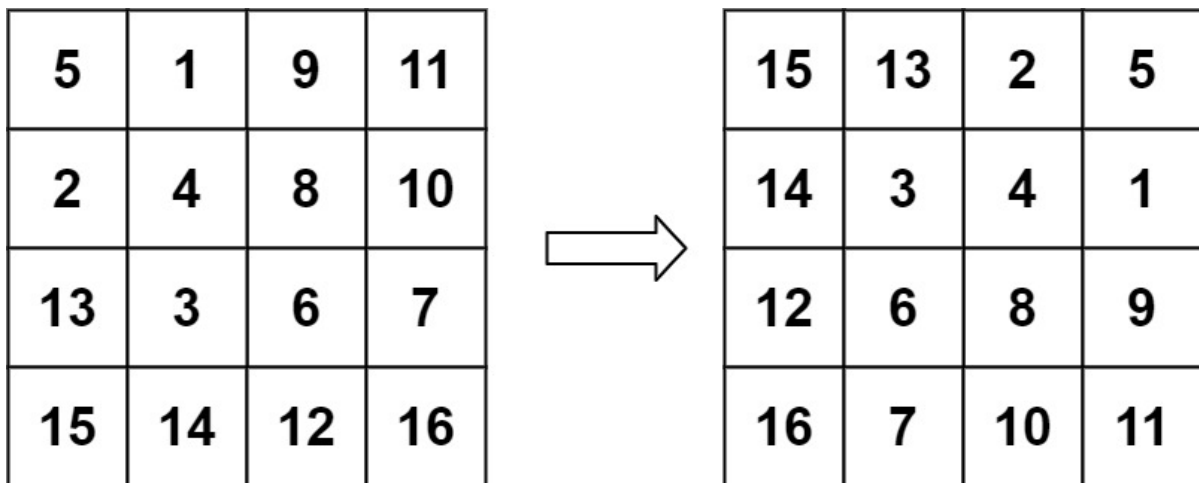
你必须在 原地 旋转图像，这意味着你需要直接修改输入的二维矩阵。**请不要** 使用另一个矩阵来旋转图像。

示例 1:



```
1 输入: matrix = [[1,2,3],[4,5,6],[7,8,9]]
2 输出: [[7,4,1],[8,5,2],[9,6,3]]
```

示例 2:



```
1 输入: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
2 输出: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]
```

提示:

- `n == matrix.length == matrix[i].length`
- `1 <= n <= 20`
- `-1000 <= matrix[i][j] <= 1000`

解析

```
1 void rotate(vector<vector<int>>& matrix) {
2     int n = matrix.size();
3     for (int i = 0; i < (n + 1) / 2; i++) {
4         for (int j = 0; j < n / 2; j++) {
5             int temp = matrix[i][j];
6             matrix[i][j] = matrix[n - 1 - j][i];
7             matrix[n - 1 - j][i] = matrix[n - 1 - i][n - 1 - j];
8             matrix[n - 1 - i][n - 1 - j] = matrix[j][n - 1 - i];
9             matrix[j][n - 1 - i] = temp;
10        }
11    }
12 }
```

我们在旋转的时候，往往可以只考虑一部分，例如这里的旋转90度，我们可以把整个矩阵分成四个部分，然后第一个部分旋转来到第二个部分，第二个部分的旋转来到第三个，由此连锁的移动就构成了旋转。

不同奇偶边数的矩阵我们可以这么来处理

行数 $(n+1)/2$ ：偶数时等于 $n/2$ ，正好一半；奇数时多一行，用来处理中间那一行的左半边（中心点不需要动）。

列数 $n/2$ ：偶数时刚好左半边；奇数时向下取整，刚好不包括正中间那一列（中间列的中心也不用动）。